



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Desktop Widget

Desktop widgets are floating, draggable components that appear on the desktop background. They provide at-a-glance information and can be positioned and scaled by the user in edit mode.

Basic Structure

A desktop widget must extend `DraggableDesktopWidget`:

```
import QtQuick
import QtQuick.Layouts
import qs.Commons
import qs.Modules.DesktopWidgets
import qs.Widgets

DraggableDesktopWidget {
    id: root

    // Plugin API (injected by PluginService)
    property var pluginApi: null

    // Widget dimensions
    implicitWidth: 200
    implicitHeight: 120

    // Your widget content here
}
```

Required Import

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

~~DraggableDesktopWidget base component.~~

```
import qs.Modules.DesktopWidgets
```

Required Properties

pluginApi

Injected by the PluginService. Provides access to plugin APIs and services.

```
property var pluginApi: null
```

Inherited Properties

The DraggableDesktopWidget base component provides these properties automatically:

screen

The ShellScreen this widget is displayed on.

```
// Inherited from DraggableDesktopWidget  
property ShellScreen screen
```

widgetData

Configuration data for this widget instance (position, scale, custom settings).

```
// Access widget-specific data  
property var widgetData: null
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Example usage  
property color textColor: (widgetData && widgetData.textColor) ? widgetData.textColor :  
Color.mOnSurface
```

widgetIndex

Index of this widget in the monitor's widget list.

```
property int widgetIndex: -1
```

isDragging

Whether the widget is currently being dragged (read-only).

```
readonly property bool isDragging
```

isScaling

Whether the widget is currently being scaled (read-only).

```
readonly property bool isScaling
```

showBackground

Whether to show the default background container. Can be overridden from widgetData.

```
property bool showBackground: true
```

widgetScale

Current scale factor of the widget. Controlled via right-click drag in edit mode.

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
property real minScale: 0.5  
property real maxScale: 3.0
```

Sizing

Desktop widgets should define their natural size:

```
DraggableDesktopWidget {  
    id: root  
  
    // Fixed size  
    implicitWidth: 200  
    implicitHeight: 120  
  
    // Or dynamic size based on content  
    implicitWidth: contentLayout.implicitWidth + Style.marginXL * 2  
    implicitHeight: contentLayout.implicitHeight + Style.marginXL * 2  
    width: implicitWidth  
    height: implicitHeight  
  
    ColumnLayout {  
        id: contentLayout  
        // ...  
    }  
}
```

Manifest Entry

Register the desktop widget in your plugin's manifest.json:

```
{  
  "id": "my-plugin",
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
  "version": "1.0.0",  
  "entryPoints": {  
    "desktopWidget": "DesktopWidget.qml"  
  }  
}
```

Using Settings

Access plugin settings via pluginApi:

```
// Get setting with fallback to default  
readonly property string message:  
  pluginApi?.pluginSettings?.message ||  
  pluginApi?.manifest?.metadata?.defaultSettings?.message ||  
  "Default Message"
```

Desktop Widget Settings

You can provide a dedicated settings UI for your desktop widget that allows users to configure per-instance options (like show/hide background, layout mode, colors) directly from the widget's settings menu.

Register a `desktopWidgetSettings` entrypoint in your manifest.json:

```
{  
  "entryPoints": {  
    "desktopWidget": "DesktopWidget.qml",  
    "desktopWidgetSettings": "DesktopWidgetSettings.qml"  
  }  
}
```

For a complete guide, see [Desktop Widget Settings](#).

Using widgetData

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

settings.

```
DraggableDesktopWidget {  
  id: root  
  
  // Custom colors from widget data  
  property color textColor: {  
    var color = widgetData && widgetData.textColor ? widgetData.textColor : ""  
    return (color && color !== "") ? color : Color.mOnSurface  
  }  
  
  // Custom font size  
  property real fontSize: {  
    var size = widgetData && widgetData.fontSize ? widgetData.fontSize : 0  
    return (size && size > 0) ? size : Style.fontSizeL  
  }  
  
  // Opacity  
  property real widgetOpacity: (widgetData && widgetData.opacity) ? widgetData.opacity : 1.0  
  
  // Boolean options  
  property bool showDetails: (widgetData && widgetData.showDetails !== undefined) ?  
  widgetData.showDetails : true  
}
```

Styling

Use Noctalia's design system for consistent styling:

Colors

```
import qs.Commons
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Primary text
color: Color.mOnSurface

// Secondary text
color: Color.mOnSurfaceVariant

// Accent color
color: Color.mPrimary
}
```

Typography

```
NText {
  pointSize: Style.fontSizeS
  pointSize: Style.fontSizeM
  pointSize: Style.fontSizeL
  pointSize: Style.fontSizeXL
  pointSize: Style.fontSizeXXL

  font.weight: Font.Light
  font.weight: Font.Normal
  font.weight: Font.Medium
  font.weight: Font.Bold
}
```

Spacing

```
ColumnLayout {  
  anchors.fill: parent
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    spacing: Style.marginS  
    spacing: Style.marginM  
    spacing: Style.marginL  
  }
```

Background Control

The `DraggableDesktopWidget` provides a default styled background with shadow. You can control it this way but it's preferred to keep as `true` if you don't add your own custom background so users have the ability to disable/enable it via the settings:

```
DraggableDesktopWidget {  
  id: root  
  
  // Disable the default background (for transparent or custom backgrounds)  
  showBackground: false  
  
  // Or let it be controlled by widget settings  
  // showBackground is automatically bound to widgetData.showBackground if available  
}
```

Edit Mode

Users can position and scale desktop widgets in edit mode.

Your widget should respond gracefully to scaling. The `isDragging` and `isScaling` properties let you adjust behavior during these operations:


```
DraggableDesktopWidget {  
  id: root
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Disable animations during drag/scale for better performance  
Rectangle {  
  opacity: root.isDragging ? 0.8 : 1.0  
  
  Behavior on opacity {  
    enabled: !root.isDragging && !root.isScaling  
    NumberAnimation { duration: 200 }  
  }  
}  
}
```

Complete Examples

Simple Message Widget

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Modules.DesktopWidgets
import qs.Widgets
```

```
DraggableDesktopWidget {
    id: root
```

```
    property var pluginApi: null
```

```
    readonly property string message:
```

```
        pluginApi?.pluginSettings?.message ||
        pluginApi?.manifest?.metadata?.defaultSettings?.message ||
        "Hello World"
```

```
    // Scale dimensions by widgetScale
```

```
    implicitWidth: Math.round(200 * widgetScale)
```

```
    implicitHeight: Math.round(120 * widgetScale)
```

```
    width: implicitWidth
```

```
    height: implicitHeight
```

```
    ColumnLayout {
```

```
        anchors.fill: parent
```

```
        anchors.margins: Math.round(Style.marginL * widgetScale)
```

```
        spacing: Math.round(Style.marginS * widgetScale)
```

```
        NIcon {
```

```
            icon: "noctalia"
```

```
            pointSize: Math.round(Style.fontSizeXXL * widgetScale)
```

```
            Layout.alignment: Qt.AlignHCenter
```

```
        }
```

```
        NText {
```

```
            text: root.message
```

```
            pointSize: Math.round(Style.fontSizeM * widgetScale)
```

```
            Layout.alignment: Qt.AlignHCenter
```

```
        }
```

```
NText {  
    text: "Desktop Widget"
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    Layout.alignment: Qt.AlignHCenter
```

```
}
```

```
}
```

```
}
```

Widget with Custom Settings

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Modules.DesktopWidgets
import qs.Widgets

DraggableDesktopWidget {
    id: root

    property var pluginApi: null

    // Customizable properties from widgetData
    property color textColor: {
        var color = widgetData && widgetData.textColor ? widgetData.textColor : ""
        return (color && color !== "") ? color : Color.mOnSurface
    }

    property real baseFontSize: {
        var size = widgetData && widgetData.fontSize ? widgetData.fontSize : 0
        return (size && size > 0) ? size : Style.fontSizeL
    }

    property real widgetOpacity: (widgetData && widgetData.opacity) ? widgetData.opacity : 1.0

    // Scale dimensions - use Math.round for pixel-perfect rendering
    implicitWidth: Math.round((contentLayout.implicitWidth + Style.marginXL * 2) * widgetScale)
    implicitHeight: Math.round((contentLayout.implicitHeight + Style.marginXL * 2) * widgetScale)
    width: implicitWidth
    height: implicitHeight

    ColumnLayout {
        id: contentLayout
        anchors.centerIn: parent
        spacing: Math.round(Style.marginM * widgetScale)
        opacity: root.widgetOpacity

        NText {
            text: pluginApi?.pluginSettings?.message || "Custom Widget"
            pointSize: Math.round(root.baseFontSize * widgetScale)
```

color: `root.textColor`

Layout.alignment: `Qt.AlignHCenter`

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NText {  
    text: new Date().toLocaleDateString()  
    pointSize: Math.round(Style.fontSizeS * widgetScale)  
    color: Color.mOnSurfaceVariant  
    Layout.alignment: Qt.AlignHCenter  
}  
}  
}
```

Transparent Widget (No Background)

```
import QtQuick
import QtQuick.Effects
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Modules.DesktopWidgets
import qs.Widgets

DraggableDesktopWidget {
    id: root

    property var pluginApi: null

    // Disable default background
    showBackground: false

    // Scale dimensions
    implicitWidth: Math.round(300 * widgetScale)
    implicitHeight: Math.round(150 * widgetScale)
    width: implicitWidth
    height: implicitHeight

    // Custom transparent styling
    ColumnLayout {
        anchors.fill: parent
        anchors.margins: Math.round(Style.marginL * widgetScale)
        spacing: Math.round(Style.marginS * widgetScale)

        NText {
            text: "12:34"
            pointSize: Math.round(Style.fontSizeXXXL * 2 * widgetScale)
            font.weight: Font.Light
            color: Color.mOnSurface
            Layout.alignment: Qt.AlignHCenter

            // Optional: add shadow for visibility on any background
            // Disable during scaling for performance
            layer.enabled: !root.isScaling
            layer.effect: MultiEffect {
                shadowEnabled: true
            }
        }
    }
}
```

shadowBlur: 0.5

shadowOpacity: 0.3

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
}  
}
```

Widget Using Services

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Modules.DesktopWidgets
import qs.Widgets
import qs.Services.Hardware

DraggableDesktopWidget {
    id: root

    property var pluginApi: null

    // Scale dimensions
    implicitWidth: Math.round(180 * widgetScale)
    implicitHeight: Math.round(100 * widgetScale)
    width: implicitWidth
    height: implicitHeight

    ColumnLayout {
        anchors.fill: parent
        anchors.margins: Math.round(Style.marginL * widgetScale)
        spacing: Math.round(Style.marginS * widgetScale)

        RowLayout {
            Layout.alignment: Qt.AlignHCenter
            spacing: Math.round(Style.marginS * widgetScale)

            NIcon {
                icon: BatteryService.icon
                pointSize: Math.round(Style.fontSizeL * widgetScale)
                color: BatteryService.charging ? Color.mPrimary : Color.mOnSurface
            }

            NText {
                text: Math.round(BatteryService.percentage) + "%"
                pointSize: Math.round(Style.fontSizeL * widgetScale)
                font.weight: Font.Bold
            }
        }
    }
}
```



```
NText {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
    color: Color.mOnSurfaceVariant
    Layout.alignment: Qt.AlignHCenter
}
}
```

Implementing Scaling

Desktop widgets support user-controlled scaling from 0.5x to 5x. **You must implement dimension-based scaling** rather than using Qt's transform-based scale property. Transform-based scaling usually cause jagged edges and overall poor rendering quality.

The widgetScale Property

The base DraggableDesktopWidget provides a widgetScale property (default: 1.0) that represents the current scale factor. Your widget must multiply all dimension-related values by this property.

What to Scale

Multiply these values by widgetScale:

1. **Widget dimensions** (implicitWidth, implicitHeight, width, height)
2. **Font sizes** (pointSize, pixelSize)
3. **Margins and spacing** (anchors.margins, Layout.margins, spacing)
4. **Border radii** (radius)
5. **Icon sizes** (pointSize for icons, baseSize for buttons)
6. **Fixed element sizes** (image containers, buttons, etc.)

Scaling Pattern

Always use `Math.round()` to ensure pixel-perfect values:

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
DraggableDesktopWidget {  
    id: root
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Scale widget dimensions  
implicitWidth: Math.round(400 * widgetScale)  
implicitHeight: Math.round(64 * widgetScale + Style.marginM * widgetScale * 2)  
width: implicitWidth  
height: implicitHeight  
  
RowLayout {  
    anchors.fill: parent  
    // Scale margins and spacing  
    anchors.margins: Math.round(Style.marginM * widgetScale)  
    spacing: Math.round(Style.marginS * widgetScale)  
  
    // Scale fixed-size elements  
    Item {  
        Layout.preferredWidth: Math.round(48 * widgetScale)  
        Layout.preferredHeight: Math.round(48 * widgetScale)  
  
        NImageRounded {  
            anchors.fill: parent  
            // Scale border radius  
            radius: Math.round(Style.radiusM * widgetScale)  
        }  
    }  
  
    NText {  
        // Scale font sizes  
        pointSize: Math.round(Style.fontSizeM * widgetScale)  
    }  
  
    NIconButton {  
        // Scale button sizes  
        baseSize: Math.round(32 * widgetScale)  
        customRadius: Math.round(Style.radiusS * widgetScale)  
    }  
}
```

Performance: Disabling Expensive Effects During Scaling

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

The `isScaling` property is `true` while the user is actively resizing the widget. Use this to disable expensive operations that would cause lag:

```
DraggableDesktopWidget {
  id: root

  // Disable layer effects during scaling (expensive GPU operations)
  Rectangle {
    layer.enabled: Settings.data.general.enableShadows && !root.isScaling
    layer.effect: MultiEffect {
      shadowEnabled: true
      shadowBlur: 0.5
    }
  }

  // Disable drop shadows during scaling
  NDropShadow {
    visible: !root.isScaling
    source: contentLayout
  }

  // Conditionally load expensive components
  Loader {
    active: shouldShowVisualizer && !root.isScaling
    sourceComponent: ExpensiveVisualizerComponent {}
  }

  // Disable animations during scaling
  Behavior on someProperty {
    enabled: !root.isScaling && !root.isDragging
    NumberAnimation { duration: 200 }
  }
}
```

What to Disable During Scaling

- **Layer effects** (layer.enabled: false or layer.enabled: ... && !root.isScaling)
- **MultiEffect shadows** (expensive GPU recomputations on every size change)

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- **Canvas/CustomPainter** operations (expensive redraws)
- **Complex Loaders** (set active: ... && !root.isScaling to completely unload)
- **Animations** (disable Behaviors during scaling)

Complete Scaling Example

```
import QtQuick
import QtQuick.Effects
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Modules.DesktopWidgets
import qs.Widgets

DraggableDesktopWidget {
    id: root

    property var pluginApi: null

    // Scaled dimensions
    implicitWidth: Math.round(300 * widgetScale)
    implicitHeight: Math.round(100 * widgetScale)
    width: implicitWidth
    height: implicitHeight

    // Drop shadow - disabled during scaling for performance
    NDropShadow {
        visible: !root.isScaling
        anchors.fill: contentLayout
        source: contentLayout
        shadowBlur: 1.0
        shadowOpacity: 0.9
    }

    ColumnLayout {
        id: contentLayout
        anchors.fill: parent
        anchors.margins: Math.round(Style.marginL * widgetScale)
        spacing: Math.round(Style.marginS * widgetScale)

        NText {
            text: "Scaled Widget"
            pointSize: Math.round(Style.fontSizeL * widgetScale)
            font.weight: Font.Bold
            Layout.alignment: Qt.AlignHCenter
        }
    }
}
```

```
RowLayout {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NIconButton {  
    baseSize: Math.round(32 * widgetScale)  
    icon: "heart"  
    customRadius: Math.round(Style.radiusS * widgetScale)  
}
```

```
NIconButton {  
    baseSize: Math.round(32 * widgetScale)  
    icon: "star"  
    customRadius: Math.round(Style.radiusS * widgetScale)  
}  
}  
}  
}
```

Best Practices

1. **Use DraggableDesktopWidget:** Always extend this base component for proper drag/scale behavior
2. **Define implicit size:** Set `implicitWidth` and `implicitHeight` for proper layout
3. **Implement dimension-based scaling:** Multiply all sizes by `widgetScale` with `Math.round()`
4. **Disable expensive effects during scaling:** Use `isScaling` to turn off layer effects, shadows, and complex components
5. **Handle widgetData:** Use defensive checks when accessing widget-specific data
6. **Respect theming:** Use `Color.m*` colors that adapt to light/dark mode
7. **Keep it performant:** Avoid expensive operations since widgets are always visible
8. **Disable animations during interaction:** Use `isDragging` and `isScaling` to optimize
9. **Provide fallbacks:** Always have default values for settings and data

Adding Desktop Widgets to the Desktop

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

1. Open Settings panel
2. Navigate to Desktop Widgets
3. Add new widget from the widget picker
4. Enable edit mode
5. Position and scale as desired
6. Exit edit mode

See Also

- [Getting Started](#) - Create your first plugin
- [Bar Widget Development](#) - Create bar widgets
- [Panel Development](#) - Create overlay panels
- [Plugin API](#) - Full API reference
- [Manifest Reference](#) - Plugin configuration